

Taller 3: Robot diferencial con brazo robótico y clasificación visual mediante cámara Raspberry Pi

Juan Sebastián Sarmiento Roncancio
Ingeniería Eléctrica y Electrónica
Universidad de los Andes
Bogotá, Colombia
js.sarmiento@uniandes.edu.co

Juan Pablo Puentes Bayona
Ingeniería Eléctrica y Electrónica
Universidad de los Andes
Bogotá, Colombia
jp.puentes@uniandes.edu.co

Juan Sebastián Hoyos Henao
Ingeniería Eléctrica y Electrónica
Universidad de los Andes
Bogotá, Colombia
js.hoyos@uniandes.edu.co

Samir David Fernández Yunis
Ingeniería Eléctrica y Electrónica
Universidad de los Andes
Bogotá, Colombia
sd.fernandez@uniandes.edu.co

Abstract—En este taller se desarrolló la integración de un sistema robótico diferencial previamente construido con un brazo robótico accionado por servomotores y un sistema de visión artificial mediante cámara Raspberry Pi y OpenCV. El objetivo principal fue lograr que el sistema identificara formas y colores básicos presentes en bloques físicos para posteriormente ejecutar una acción de agarre mediante el brazo robótico. Durante la implementación experimental se logró una detección visual estable de figuras geométricas básicas y colores primarios, observándose un funcionamiento sólido del sistema de visión artificial. Sin embargo, aunque el brazo robótico presentó movimiento funcional, no alcanzó la capacidad mecánica necesaria para sujetar y levantar bloques de dimensiones $15 \times 15 \times 15$ cm, principalmente debido a limitaciones de torque en los servomotores utilizados.

Index Terms—ROS2, visión artificial, HSV, robot diferencial, brazo robótico, OpenCV, Raspberry Pi.

I. INTRODUCCIÓN

Este taller corresponde a la continuación del sistema robótico desarrollado previamente, incorporando nuevas capacidades de percepción visual y manipulación mecánica. Sobre la plataforma móvil ya existente se integró una cámara conectada directamente a la Raspberry Pi mediante interfaz CSI, y un brazo robótico de dos grados de libertad controlado mediante servomotores MG996R.

El propósito general consistió en detectar automáticamente el color y la forma de bloques ubicados frente al robot, permitiendo posteriormente una acción mecánica de clasificación mediante el brazo. Para la comunicación entre subsistemas se utilizó ROS2 Jazzy como middleware, ejecutándose sobre Ubuntu 24.04 en una Raspberry Pi 5.

Aunque la etapa de percepción visual alcanzó resultados satisfactorios, la manipulación física presentó limitaciones mecánicas importantes que condicionaron el desempeño global del sistema.

II. MARCO TEÓRICO

El sistema de manipulación implementado corresponde a un brazo robótico de dos grados de libertad activos compuesto por

servomotores MG996R. La detección visual se implementó mediante OpenCV usando análisis de contornos, clasificación geométrica y detección de color en espacio HSV.

A. Espacio de Color HSV

El espacio HSV (*Hue, Saturation, Value*) separa la información cromática de la luminosidad, lo que lo hace más robusto que RGB ante variaciones de iluminación. El canal H define el tono del color, S la saturación y V el brillo, permitiendo definir rangos de color estables con independencia de las condiciones lumínicas del entorno.

El color rojo requiere dos rangos HSV debido a que se encuentra en los extremos del espectro de matiz ($H \approx 0^\circ$ y $H \approx 180^\circ$), mientras que verde y azul se definen con rangos continuos.

B. Clasificación Geométrica por Contornos

La clasificación de formas se realiza mediante aproximación poligonal del contorno. El algoritmo de Ramer-Douglas-Peucker reduce el contorno a su representación poligonal mínima, permitiendo contar los vértices resultantes. Un contorno con cuatro vértices y relación ancho/alto cercana a 1 corresponde a un cuadrado, que es la forma de los cubos a clasificar.

C. Servomotores MG996R

Los servomotores MG996R tienen un torque nominal de 9.4 kg·cm a 4.8 V y 11 kg·cm a 6 V. Este parámetro es crítico ya que el torque efectivo en la articulación depende no solo de la capacidad del servo sino también de la longitud del brazo de palanca. Con brazos extendidos horizontalmente, el torque requerido puede superar fácilmente la capacidad nominal del actuador.

III. DISEÑO MECÁNICO

El brazo robótico fue concebido como un mecanismo de manipulación tipo montacarga compuesto por una estructura

TABLE I
MATERIALES DEL BRAZO ROBÓTICO

Componente	Cant.	Función
Servomotor MG996R	2	Articulaciones principales
Estructura impresa 3D	1	Armazon del brazo
Paletas montacarga	1 set	Agarre inferior del cubo
Tornillería M3	variable	Ensamble estructural

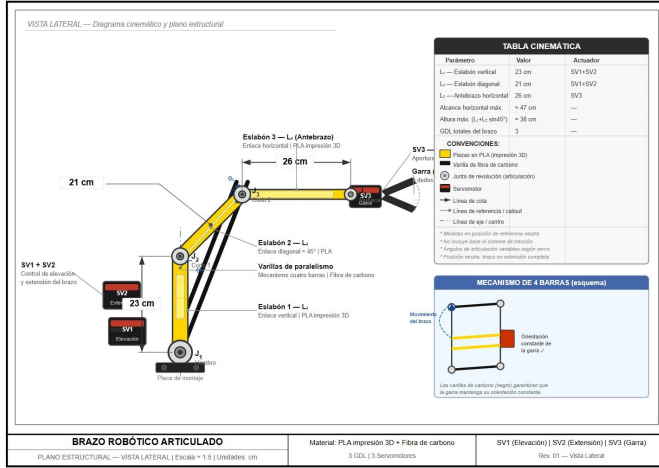


Fig. 1. Diagrama cinemático y plano estructural del brazo robótico — vista lateral. Escala $\approx 1:5$. Unidades: cm.

articulada accionada mediante servomotores, cuyo objetivo principal era aproximarse a los bloques y ejecutar una acción de agarre mediante paletas ubicadas en el extremo del brazo.

A. Diseño General

El brazo cuenta con dos grados de libertad principales: un servo que controla la altura (elevación/descenso del conjunto) y un servo que controla la extensión horizontal. Las medidas aproximadas del sistema son: desde las paletas hasta el primer servomotor, aproximadamente 26 cm; segmento medio inclinado, aproximadamente 21 cm; y parte vertical de base, aproximadamente 23 cm.

B. Materiales Utilizados

Los materiales empleados en la construcción del brazo se resumen en la Tabla I. La estructura fue fabricada en su totalidad mediante impresión 3D, lo que permitió ajustar la geometría de los eslabones de forma iterativa durante el proceso de diseño.

C. Planos Mecánicos

La Fig. 1 presenta el diagrama cinemático y plano estructural del brazo robótico en vista lateral. El mecanismo de cuatro barras con varillas de fibra de carbono garantiza que la garra mantenga una orientación constante durante todo el movimiento de elevación y extensión.

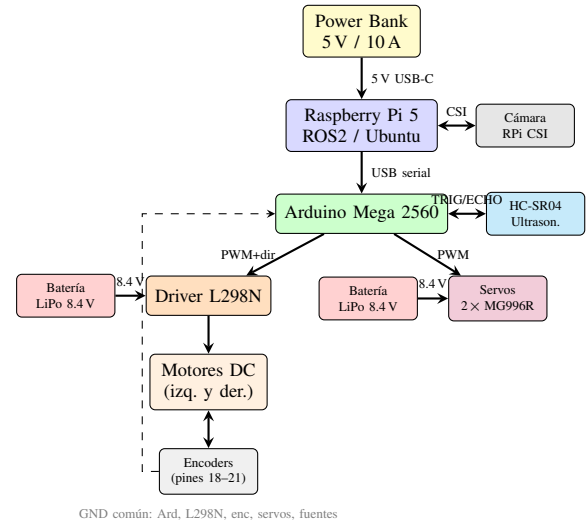


Fig. 2. Plano electrónico general del sistema robótico.

D. Funcionalidad Observada

El brazo robótico fue capaz de ejecutar movimientos básicos mediante los dos servomotores, permitiendo desplazamientos angulares controlados. La posición HOME corresponde a 180° en el servo de altura (brazo elevado), y la posición de recogida corresponde a 0° (paletas al nivel del suelo).

Sin embargo, durante la sustentación experimental se evidenció que el sistema no alcanzó la fuerza suficiente para sujetar correctamente el bloque asignado. La limitación principal se observó en la fase de elevación, donde las paletas lograban posicionarse bajo el cubo pero no generaban el torque necesario para levantarlo de forma estable.

IV. DISEÑO ELECTRÓNICO

La arquitectura electrónica base se mantuvo igual a la implementada en el taller anterior, conservando la Raspberry Pi 5 como unidad principal de procesamiento, el Arduino Mega 2560 como controlador auxiliar y el sistema de potencia previamente definido.

A. Nuevas Incorporaciones

Las adiciones respecto al taller anterior fueron: la cámara Raspberry Pi conectada directamente mediante interfaz CSI al puerto CAM0; los servomotores del brazo robótico controlados desde el Arduino Mega a través de los pines 2 y 3; y el sensor ultrasonico HC-SR04 conectado a los pines 30 (TRIG) y 31 (ECHO) del Arduino, cuya lectura se transmite vía serial a ROS2.

B. Diagrama Electrónico General

La Fig. 2 presenta el plano electrónico general del sistema, destacando las cuatro fuentes de alimentación independientes y los flujos de señal entre subsistemas.

C. Tabla de Conexiones

La Tabla II detalla la asignación de pines del Arduino Mega para todos los periféricos del sistema.

TABLE II
CONEXIONES DE PINES DEL ARDUINO MEGA

Elemento	Pin / Cable	Conectado a
L298N IN1	Pin 8	Arduino Mega
L298N IN2	Pin 9	Arduino Mega
L298N ENA	Pin 10 (PWM)	Arduino Mega
L298N IN3	Pin 11	Arduino Mega
L298N IN4	Pin 12	Arduino Mega
L298N ENB	Pin 13 (PWM)	Arduino Mega
Encoder Motor S1	Pin 18	Arduino Mega
Encoder Motor S2	Pin 19	Arduino Mega
Encoder Motor S1	Pin 20	Arduino Mega
Encoder Motor S2	Pin 21	Arduino Mega
Servo 1 (altura)	Pin 2	Arduino Mega
Servo 2 (extensión)	Pin 3	Arduino Mega
HC-SR04 TRIG	Pin 30	Arduino Mega
HC-SR04 ECHO	Pin 31	Arduino Mega
Raspberry Pi 5	USB-A	Arduino USB-B

TABLE III
SISTEMA DE ALIMENTACIÓN ELÉCTRICA DEL ROBOT

Fuente	V	I	Alimenta
Bater. LiPo 1	8.4 V	2900 mAh	Motores DC
Bater. LiPo 2	8.4 V	2900 mAh	Servos MG996R
Power Bank	5 V	10 A	Raspberry Pi 5
Raspberry Pi 5	5 V	≤500 mA	Arduino Mega

D. Sistema de Alimentación Eléctrica

La Tabla III resume las cuatro fuentes de alimentación independientes empleadas. Los servomotores MG996R consumen hasta 2.5 A bajo carga, por lo que nunca deben alimentarse desde los pines 5 V del Arduino ni de la Raspberry Pi; la separación de fuentes es crítica para evitar reinicios inesperados.

V. INTEGRACIÓN MECÁNICA Y ELECTRÓNICA

Esta sección describe explícitamente la relación entre los componentes mecánicos y electrónicos del sistema, evidenciando cómo la electrónica habilita el movimiento y la clasificación de objetos.

A. Relación Actuadores–Estructura Mecánica

Los dos servomotores MG996R están montados directamente sobre la base impresa en 3D del brazo robótico. El **SV1 (Elevación)** está fijado en la articulación J_1 (hombro) y controla el giro del eslabón vertical L_1 de 23 cm; el **SV2 (Extensión)** opera en J_2 (codo) y gobierna el eslabón diagonal L_2 de 21 cm. Ambos reciben señales PWM directamente desde los pines 2 y 3 del Arduino Mega, sin etapa de

potencia intermedia, dado que se alimentan desde una batería LiPo dedicada de 8.4 V. El mecanismo de cuatro barras con varillas de fibra de carbono conecta ambas articulaciones de forma que la orientación de las paletas se mantiene constante independientemente del ángulo de elevación, lo que es crítico para el agarre tipo montacarga.

Los dos motores DC de tracción están acoplados mecánicamente a las ruedas mediante encoders de cuadratura. El driver L298N recibe las señales de dirección (IN1–IN4) y habilitación PWM (ENA, ENB) desde el Arduino Mega, y entrega la corriente necesaria a los motores desde la primera batería LiPo. Los encoders realimentan la posición angular de cada rueda al sistema de odometría, cerrando el lazo de estimación de postura.

B. Interacción Sensores–Sistema de Control

El sensor ultrasonico HC-SR04 está montado en la parte frontal del chasis, orientado en la dirección de avance del robot. El Arduino Mega genera el pulso de disparo (TRIG, pin 30) y mide el tiempo de vuelo del eco (ECHO, pin 31) para calcular la distancia al objeto. Esta medición se transmite por serial al `serial_bridge_node` de ROS2, que la publica en `/taller3/ultrasonido`. El `vision_node` suscribe a ese tópic y lo emplea como condición de parada en el lazo de centrado visual: solo cuando $d \leq 30$ cm y $|e_x| \leq 80$ px se activa la secuencia de agarre, garantizando que el brazo esté físicamente al alcance del cubo antes de intentar recogerlo.

La cámara Raspberry Pi, conectada por CSI directamente a la Raspberry Pi 5, proporciona la realimentación visual que determina los comandos de velocidad angular del robot. De esta forma, la percepción visual sustituye a un sensor de posición absoluta: el robot se orienta automáticamente hacia el cubo detectado hasta alcanzar la condición de proximidad medida por el ultrasonido.

C. Cadena Completa: Percepción → Decisión → Actuación

La Fig. 2 ilustra el flujo de señales del sistema. La cadena de control completa opera así: (1) la cámara captura el frame y el `vision_node` detecta el cubo; (2) se calcula el error de centrado e_x y se consulta la distancia del ultrasonido; (3) el `vision_node` publica en `/cmd_vel` el comando de giro o en `/taller3/comando_robot` el comando de brazo; (4) el `serial_bridge_node` traduce esos mensajes ROS2 a cadenas seriales hacia el Arduino; (5) el Arduino Mega ejecuta los comandos sobre el L298N (motores) o los servos (brazo). La Raspberry Pi 5 actúa como cerebro central que integra percepción, decisión y comunicación, mientras el Arduino Mega descarga la generación de señales PWM en tiempo real y la lectura del ultrasonido.

D. Cinemática del Sistema

1) *Cinemática diferencial de la plataforma móvil*: El robot utiliza una configuración diferencial con dos ruedas motrices. La Raspberry Pi calcula la odometría a partir de los pulsos de los encoders mediante las ecuaciones:

TABLE IV
NODOS ROS2 DEL PAQUETE TALLER3_GRUPO2

Nodo	Función principal
encoder_node	Lee encoders de ruedas por GPIO RPi
odometry_node	Cinemática diferencial; publica /odom
serial_bridge_node	Puente ROS2 ↔ Arduino 115200 baud
vision_node	Detección cubos HSV + control visual
recorder_node	Graba recorridos en archivos .txt
player_node	Reproduce recorridos (/play_route)
teleop_node	Control manual W/A/S/D
web_dashboard_node	Dashboard web en puerto 8081

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{b} \quad (1)$$

donde v_R y v_L son las velocidades lineales de la rueda derecha e izquierda respectivamente, y b es la distancia entre ruedas (track). La postura del robot (x, y, θ) se integra incrementalmente en cada periodo de muestreo:

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{pmatrix} = \begin{pmatrix} x_k + v \Delta t \cos \theta_k \\ y_k + v \Delta t \sin \theta_k \\ \theta_k + \omega \Delta t \end{pmatrix} \quad (2)$$

Esta estimación es publicada continuamente en /odom y /robot_pose por el odometry_node.

2) *Cinemática del brazo robótico*: El brazo articulado tiene 2 GDL activos (θ_1 : elevación, θ_2 : extensión) más el eslabón terminal de 26 cm (L_3) controlado por SV3. La posición del extremo en el plano vertical puede expresarse mediante cinemática directa como:

$$x_e = L_3 + L_2 \cos \theta_2, \quad z_e = L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \quad (3)$$

El alcance horizontal máximo teórico es de ≈ 47 cm y la altura máxima de ≈ 38 cm, según los parámetros de la Tabla Cinemática del plano estructural (Fig. 1). En la implementación actual, los ángulos se comandan directamente en grados a los servos (posición HOME: $\theta_1 = 180^\circ$; posición de recogida: $\theta_1 = 0^\circ$), sin lazo cerrado de posición.

VI. ARQUITECTURA ROS2

ROS2 Jazzy fue utilizado como middleware de comunicación entre la Raspberry Pi 5 y los distintos subsistemas del robot. El paquete desarrollado se denomina taller3_grupo2, ubicado en `~/ros2_ws/src/taller3_grupo2/`, e implementa ocho nodos independientes con responsabilidades bien delimitadas.

A. Nodos Implementados

La Tabla IV presenta el resumen de los ocho nodos. A continuación se describe en detalle la responsabilidad y comportamiento de cada uno.

El nodo `encoder_node` lee directamente los pines GPIO de la Raspberry Pi (pines 17 y 27) conectados a los encoders de cuadratura de ambas ruedas y publica los conteos acumulados en /wheel_ticks. El nodo `odometry_node`

TABLE V
TÓPICOS ROS2 DEL SISTEMA

Tópico	Tipo msg	Publicador
/cmd_vel	Twist	teleop/vision
/wheel_ticks	Int32MultiArray	encoder_node
/odom	Odometry	odometry_node
/robot_pose	Float32MultiArray	odometry_node
/taller3/objeto_detectado	String	vision_node
/taller3/ultrasonido	Float32	serial_bridge
/taller3/comando_robot	String	vision/dash.
/taller3/imagen_procesada	Image	vision_node

consume ese tópico, aplica el modelo cinemático diferencial y publica tanto el mensaje estándar /odom (nav_msgs) como /robot_pose (Float32MultiArray con x, y, θ) consumido por el dashboard.

El nodo `serial_bridge_node` actúa como pasarela bidireccional entre ROS2 y el Arduino Mega a través del puerto /dev/ttyACM0 a 115200 baudios. En dirección descendente, traduce los mensajes Twist recibidos en /cmd_vel a la cadena de comandos que el L298N espera, y retransmite los comandos de brazo recibidos en /taller3/comando_robot al firmware del Arduino. En dirección ascendente, parsea la línea serial del Arduino para extraer la medición del ultrasonido HC-SR04 y publicarla en /taller3/ultrasonido. El nodo implementa reconexión automática ante desconexiones del puerto serial y requiere permisos de acceso (`chmod 666 /dev/ttyACM0`) al inicio de la sesión.

El nodo `teleop_node` captura pulsaciones de teclado en tiempo real: W avanza, S retrocede, A y D giran a izquierda y derecha respectivamente, y la barra espaciadora detiene el robot. Los comandos se publican en /cmd_vel con las mismas velocidades escaladas que usa el nodo de visión, garantizando coherencia en el comportamiento del robot independientemente del modo de control. El nodo `web_dashboard_node` expone una interfaz HTTP en el puerto 8081 que agrega la imagen procesada, la posición odometrada, el estado de detección y controles básicos de teleop, permitiendo operar el robot desde cualquier PC en la misma red sin necesidad de monitor en la Raspberry Pi.

El nodo `recorder_node` graba en archivos .txt los comandos de velocidad ejecutados junto con su marca de tiempo, y el nodo `player_node` los reproduce a través del servicio /play_route (interfaz PlayRoute.srv definida en el paquete taller3_interfaces), permitiendo la ejecución repetible de trayectorias predefinidas.

B. Tópicos ROS2

La Tabla V presenta los tópicos principales del sistema. Tanto `teleop_node` como `vision_node` publican en /cmd_vel, y el `serial_bridge_node` es el único suscriptor de ese tópico, lo que garantiza una interfaz única hacia el hardware sin modificar el firmware del Arduino.

C. Estructura del Sistema

El flujo de información principal sigue las cadenas mostradas a continuación. La separación de trayectorias permite que el nodo de visión y el operador humano compitan por `/cmd_vel` de forma transparente:

```
teleop_node -> /cmd_vel -> serial_bridge -> Arduino
vision_node -> /cmd_vel -> serial_bridge -> Arduino
vision_node -> /comando_robot -> serial_bridge -> Brazo
Arduino -> serial -> serial_bridge -> /ultrasonido
encoder_node -> /wheel_ticks -> odometry_node
```

VII. PROCESAMIENTO DE IMAGEN CON LA CÁMARA RASPBERRY PI

La cámara Raspberry Pi (módulo conectado al puerto CSI CAM0) captura video en formato MJPEG a 640×480px a 30fps mediante el comando `rpivid`. El proceso se ejecuta como subproceso del nodo `vision_node`, que lee los frames JPEG desde el `stdout` del subproceso mediante `subprocess.Popen` y los decodifica con `cv2.imdecode`. La imagen procesada con anotaciones visuales (bounding box, color, área, error e_x) se publica en `/taller3/imagen_procesada` y adicionalmente se sirve por HTTP en el puerto 8082, accesible desde cualquier navegador sin requerir RViz ni herramientas ROS en el PC de monitoreo.

A. Pipeline de Detección

El pipeline completo de detección opera sobre cada frame de la siguiente manera:

- 1) **Suavizado Gaussiano** (kernel 7×7): reduce el ruido de alta frecuencia antes de la segmentación por color.
- 2) **Conversión BGR → HSV**: desacopla la crominancia de la luminancia para hacer la segmentación robusta ante cambios de iluminación ambiental.
- 3) **Máscaras por color**: se aplican rangos HSV independientes para cada color objetivo (Tabla VI). El rojo requiere la unión (`cv2.bitwise_or`) de dos máscaras por cruzar $H = 0^\circ$.
- 4) **Morfología**: apertura seguida de cierre con kernel elíptico 9×9, para eliminar artefactos pequeños y rellenar huecos internos en las regiones segmentadas.
- 5) **Extracción de contornos**: `cv2.findContours` con modo `RETR_EXTERNAL` obtiene únicamente los contornos más externos, evitando contornos anidados de reflejos o sombras.
- 6) **Filtrado geométrico**: se descartan contornos cuya área esté fuera del rango $[2000, 200\,000] \text{ px}^2$ o cuya relación ancho/alto sea menor que 0.4 o mayor que 2.0.
- 7) **Clasificación de forma**: `cv2.approxPolyDP` con $\epsilon = 0.04 \cdot P$ reduce el polígono a sus vértices esenciales. Un contorno con ≥ 4 vértices y relación ancho/alto en $[0.75, 1.33]$ se clasifica como cuadrado/cubo.
- 8) **Color dominante**: dentro del bounding box del objeto detectado se cuenta el número de píxeles activos de cada máscara de color; el color con mayor recuento es asignado al objeto.

TABLE VI
RANGOS HSV CALIBRADOS PARA CADA COLOR

Color	H [min,max]	S [min,max]	V [min,max]
Rojo	[0, 12]	[100, 255]	[50, 255]
Rojo	[160, 180]	[100, 255]	[50, 255]
Verde	[30, 95]	[60, 255]	[40, 255]
Azul	[85, 135]	[60, 255]	[40, 255]

- 9) **Confirmación temporal**: la detección se declara válida únicamente si en 4 frames consecutivos al menos el 75 % reporta el mismo color y posición, eliminando falsos positivos por reflejos especulares y cambios de iluminación transitorios.

B. Detección de Colores

Los rangos HSV fueron calibrados empíricamente bajo las condiciones de iluminación del laboratorio. La Tabla VI los detalla. El color dominante se determina contando los píxeles activos de cada máscara dentro del bounding box del objeto y seleccionando el de mayor recuento.

C. Lógica de Control Visual

Una vez confirmada la detección de un cubo, el `vision_node` calcula el error horizontal e_x como la diferencia entre la coordenada x del centroide del objeto y el centro geométrico de la imagen ($x_c = 320 \text{ px}$). Según el valor de e_x y la distancia medida por el sensor ultrasonico d , se generan los siguientes comandos en `/cmd_vel`:

- $e_x > +80 \text{ px}$ (cubo a la derecha): $v = 0$, $\omega = -0.4 \text{ rad/s}$ → gira a la derecha.
- $e_x < -80 \text{ px}$ (cubo a la izquierda): $v = 0$, $\omega = +0.4 \text{ rad/s}$ → gira a la izquierda.
- $|e_x| \leq 80 \text{ px}$ y $d \leq 30 \text{ cm}$: el robot se detiene y activa la secuencia `ARM_PICK`.

La tolerancia `TOLERANCIA_CENTRO = 80 px` fue seleccionada empíricamente como el mínimo valor que permite al brazo alcanzar el cubo sin requerir un ajuste fino adicional. La distancia límite de 30 cm fue determinada como el rango efectivo del brazo tipo montacarga en posición extendida.

D. Secuencia ARM_PICK y Control del Brazo

Cuando se alcanza la condición de activación, el `vision_node` publica la cadena `ARM_PICK` en `/taller3/comando_robot`. El `serial_bridge_node` recibe este comando y lo transmite al Arduino por el puerto serial. El firmware del Arduino ejecuta la siguiente secuencia:

- 1) Sube el brazo a la posición HOME (180°) para asegurar el despeje sobre el cubo.
- 2) Baja el servo de altura a 0° (paletas al nivel del suelo).
- 3) Espera 5 segundos para estabilización mecánica y posicionamiento bajo el cubo.
- 4) Sube nuevamente a HOME (180°), intentando elevar el cubo.

Al completar la secuencia, el Arduino responde con el mensaje `PICK_OK` por serial, que el `serial_bridge_node` procesa para actualizar el estado interno. El servo de extensión se mantiene fijo en 90° durante toda la operación. El streaming del video procesado permanece activo durante la ejecución del brazo, permitiendo verificar visualmente el resultado desde el dashboard.

E. Modalidades de Acceso al Video

El sistema ofrece tres modalidades independientes de acceso al video:

- `ver_camara.py`: script sin ROS2 que ejecuta el pipeline HSV e imprime resultados en consola; útil para calibración rápida de rangos HSV sin necesidad de lanzar el stack completo.
- `stream_server.py` (puerto 8080): servidor HTTP que retransmite el video crudo de la cámara sin procesamiento, para verificar la conexión física del módulo CSI.
- `stream_ros.py` (puerto 8082): servidor HTTP que retransmite la imagen procesada con bounding boxes desde `/taller3/imagen_procesada`, permitiendo monitoreo en tiempo real sin pantalla en la Raspberry Pi.

VIII. RESULTADOS EXPERIMENTALES

A. Sistema de Visión

El sistema de visión artificial funcionó correctamente durante la práctica. La detección de cubos rojo, azul y verde fue estable bajo las condiciones de iluminación del laboratorio, con áreas detectadas entre 5000 y 160 000 px^2 dependiendo de la distancia al objeto. La consola imprimía en tiempo real el color, área, coordenadas del centroide, error e_x y distancia ultrasónica. El mecanismo de confirmación temporal (4 frames, 75 % de coincidencia) eliminó efectivamente los falsos positivos generados por reflejos y cambios de iluminación transitoria.

B. Sistema de Navegación

El robot respondió correctamente a los comandos de teleoperación. Sin embargo, el centrado automático basado en visión no logró posicionar el robot de forma consistente frente al cubo. Las oscilaciones en el lazo de control visual impedían una convergencia estable, principalmente por la latencia del procesamiento de imagen y la publicación en ROS2 (≈ 500 ms por ciclo).

C. Sistema de Manipulación

El brazo robótico no logró levantar los cubos durante la sustentación. Las causas identificadas fueron el torque insuficiente de los servomotores MG996R con el brazo extendido horizontalmente, ya que el brazo de palanca aumenta el torque requerido más allá de la capacidad nominal de los actuadores, y la geometría inadecuada de las paletas tipo montacarga, que no lograron un apoyo correcto bajo el cubo de $15 \times 15 \times 15$ cm.

A nivel de software, la secuencia `ARM_PICK` funcionó correctamente: los comandos se enviaban por serial, el Arduino

TABLE VII
RESUMEN DE RESULTADOS EXPERIMENTALES

Subsistema	Resultado
Visión (detección cubos)	✓ Funciona bien
Teleop y movimiento	✓ Funciona
Odometría	✓ Funciona
Dashboard web	✓ Funciona
Grabación/reproducción	✓ Funciona
Serial Arduino	✓ Funciona
Centrado visual automático	~ Oscilaciones
Brazo robótico (software)	✓ Secuencia OK
Brazo robótico (físico)	× No levanta cubo

los ejecutaba (confirmado por mensajes `PICK_OK`) y el brazo realizaba el movimiento programado (bajar a 0° , esperar, subir a 180°).

D. Integración ROS2

Los ocho nodos del paquete se lanzaron exitosamente mediante el archivo `robot_launch.py`. La comunicación serial con el Arduino se estableció a 115200 baudios con reconexión automática ante desconexiones. La odometría se calculó en tiempo real y el dashboard web fue accesible desde cualquier PC en la misma red. La Tabla VII resume el estado de todos los subsistemas.

IX. DISCUSIÓN

La principal limitación observada fue la insuficiencia de torque de los servomotores. Aunque el brazo ejecutaba movimientos controlados, no alcanzó la fuerza mecánica necesaria para levantar bloques de $15 \times 15 \times 15$ cm. La arquitectura tipo montacarga es conceptualmente correcta para este tipo de cubo, pero requiere actuadores de mayor torque (≥ 20 kg-cm) y paletas con mayor superficie de apoyo. Una solución viable sería usar servos de mayor capacidad o implementar un sistema de reducción mecánica.

Respecto al control de navegación, la latencia del lazo visión-acción (≈ 500 ms) genera oscilaciones que impiden el centrado estable. Una mejora directa sería reemplazar el control *on/off* actual por un controlador PID sobre el error e_x , lo que permitiría convergencia suave sin sobreimpulso.

Todos los subsistemas de software presentaron comportamiento sólido: visión, comunicación serial, odometría, grabación/reproducción de recorridos y dashboard web funcionaron según lo esperado.

X. CONCLUSIONES

- El sistema de visión artificial con OpenCV y espacios de color HSV demostró ser robusto para la detección de cubos en condiciones de laboratorio. La confirmación temporal por múltiples frames fue clave para eliminar detecciones erróneas.
- La arquitectura de 8 nodos ROS2 cumple los requerimientos del taller. La separación de responsabilidades facilitó el desarrollo, la depuración y la modularidad del sistema.

- La principal limitación fue mecánica: los servomotores MG996R no generaron el torque necesario con el brazo extendido. Se recomienda usar servos de ≥ 20 kg·cm o reducir el brazo de palanca.
- El sistema de alimentación con cuatro fuentes independientes fue esencial para evitar reinicios inesperados y garantizar estabilidad operacional.
- La comunicación serial bidireccional Raspberry Pi–Arduino funcionó confiablemente a 115200 baudios, permitiendo el control de actuadores y la lectura del sensor ultrasonico en tiempo real.

XI. TRABAJO FUTURO

- Implementar servomotores de mayor torque (≥ 20 kg·cm) para la articulación principal.
- Mejorar la geometría del mecanismo de agarre (paletas con mayor superficie de apoyo).
- Implementar un controlador PID para el centrado visual del robot.
- Integrar clasificación automática completa con zonas de destino diferenciadas por color.
- Agregar encoder de posición en los servos para control de lazo cerrado del brazo.

REFERENCES

- [1] Raspberry Pi Foundation, “Camera software,” *Raspberry Pi Documentation*, 2025.
- [2] OpenCV, “Open Source Computer Vision Library,” *OpenCV Docs*, 2025.
- [3] Open Robotics, “ROS 2 Documentation: Jazzy,” *Open Robotics*, 2025.
- [4] Arduino, “Arduino Mega 2560 Datasheet,” *Arduino Documentation*, 2025.
- [5] M. Pasan, “How to Setup Raspberry Pi Camera Module 3 on Ubuntu 24.04,” *DEV Community*, 2024.
- [6] Universidad de los Andes, “Guía Taller 3,” *IELE 3338 L*, 2026.